Part A — Classical ML from scratch

ml-ladder writeup · part A

2026-09-24

Contents

A1 — Linear & logistic regression by gradient descent **2**
- The idea . . . 2
- The math . . . 2
- What we measure, and why . . . 4
- What I built . . . 4
- Results . . . 4
- What surprised me . . . 5
- Open threads . . . 5
- Further reading . . . 5

A2 — Decision tree and random forest **7**
- The idea . . . 7
- The math . . . 7
- What we measure, and why . . . 8
- What I built . . . 9
- Results . . . 9
- What surprised me . . . 9
- Open threads . . . 11
- Further reading . . . 11

A3 — k-means and PCA on one dataset **11**
- The idea . . . 11
- The math . . . 12
- What we measure, and why . . . 13
- What I built . . . 14
- Results . . . 14
- What surprised me . . . 14
- Open threads . . . 14
- Further reading . . . 17

A4 — Gradient boosting, benchmarked against XGBoost **17**
- The idea . . . 17
- The math . . . 17
- What we measure, and why . . . 18
- What I built . . . 19
- Results . . . 19
- What surprised me — and a claim I had to retract . . . 19
- Open threads . . . 21

Further reading 21

What part A settled **21**

Status: complete (A1–A4) · **Constraint:** numpy only, no sklearn
Numbers: seeds 0–2, reference seed 0 · raw values in `data/a-metrics.json`
Reproduce: `ladder run A1` ... `A4`, or `uv run python tools/writeup_figures.py A`

Four algorithms, written from the loss function up, on synthetic data generated by `mlladder.datasets` — also numpy-only, for the same reason the rungs are.

The through-line: **every one of these methods is a squared-error or log-loss objective plus a search strategy, and what separates them is how they spend variance.** Gradient descent spends it on step noise, a deep tree on splits, a forest averages it away, boosting trades it against bias one shrunken step at a time. That is the frame the later parts inherit — a transformer is a different function class, not a different bargain.

Notation throughout: $X \in \mathbb{R}^{n \times d}$ is the design matrix, $y \in \mathbb{R}^n$ the targets, $X_c = X - \mathbf{1}\bar{x}^\top$ the centred design, and $\|\cdot\|$ the Euclidean norm.

A1 — Linear & logistic regression by gradient descent

Question. Linear regression has a closed form. So what does gradient descent actually cost you, and how does the *batch schedule* — full batch, mini-batch, single sample — change the answer?

The idea

Linear regression predicts a number as a weighted sum of the inputs, $\hat{y} = w^\top x + b$: one weight per feature saying how much that feature pushes the prediction up or down, plus an intercept. **Logistic regression** keeps exactly the same weighted sum but squashes it through the sigmoid $\sigma(z) = 1/(1 + e^{-z})$, so the output is a probability between 0 and 1 and the model becomes a classifier whose decision boundary is the straight line (hyperplane) $w^\top x + b = 0$.

Both are fitted the same way: define a loss that says how wrong the current weights are, then repeatedly nudge the weights *downhill* along the gradient of that loss. The nudge size is the **learning rate** η. The only design choice left is how much data to look at before each nudge:

- **full batch** — average the gradient over every training example. Each step is exact, but expensive.
- **mini-batch** — average over a random handful (32 here). Cheap, slightly noisy.
- **SGD** — one example per step. Cheapest, noisiest.

Linear regression also has a closed-form answer (the *normal equation*), which makes it the perfect test bed: the optimum is known exactly, so we can measure how far each schedule actually gets from it.

The picture the math below proves. All three start at the same point on the same bowl-shaped loss. Full-batch descent (blue) glides to the optimum. Constant- step SGD (red) arrives fast, then keeps jittering inside a ball around it, because every single-example gradient is a noisy estimate of the true one. Shrink the step over time (green) and the jitter dies out.

The math

Least squares. With $f(x) = w^\top x + b$ and an L2 penalty on the weights only,

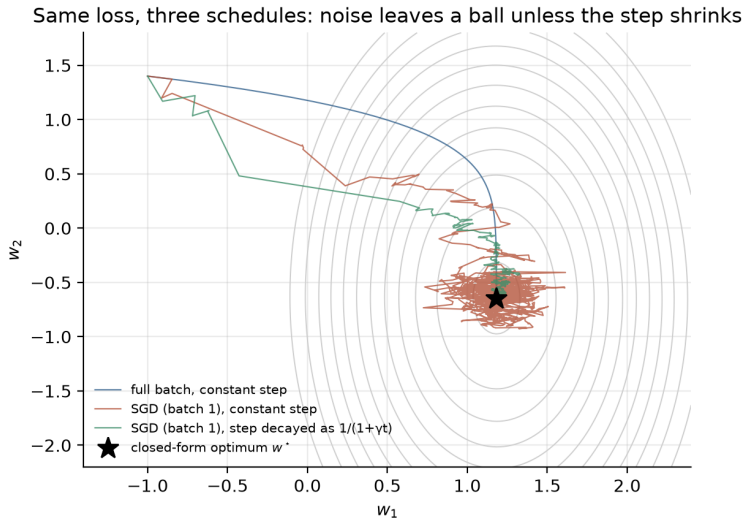


Figure 1: Gradient descent, three schedules, on one 2-D loss

$$\mathcal{L}(w, b) = \tfrac{1}{n}\|Xw + b\mathbf{1} - y\|^2 + \lambda\|w\|^2,$$

$$\nabla_w \mathcal{L} = \tfrac{2}{n}X^\top(Xw + b\mathbf{1} - y) + 2\lambda w, \qquad \partial_b \mathcal{L} = \tfrac{2}{n}\mathbf{1}^\top(Xw + b\mathbf{1} - y).$$

Setting both to zero and eliminating b gives the closed form on centred data,

$$w^\star = \left(X_c^\top X_c + \lambda n I\right)^{-1} X_c^\top (y - \bar{y}\mathbf{1}), \qquad b^\star = \bar{y} - \bar{x}^\top w^\star.$$

The intercept equation $\partial_b \mathcal{L} = 0$ says exactly *the residuals have mean zero*, which is why b absorbs the level of y. Penalizing it instead would give $b^\star = (\bar{y} - \bar{x}^\top w^\star)/(1 + \lambda')$ — an estimator that shrinks predictions toward the origin and is not equivariant under shifting y, which is why it is excluded from the penalty here.

Why the schedule matters. The Hessian is $H = \frac{2}{n}X^\top X + 2\lambda I$, with eigenvalues in $[\mu, L]$. Full-batch gradient descent is the linear recurrence

$$w_{t+1} - w^\star = (I - \eta H)(w_t - w^\star),$$

so the error contracts by $\rho(\eta) = \max\{|1 - \eta\mu|,\ |1 - \eta L|\}$ — convergent iff $0 < \eta < 2/L$, and at best $\rho = \frac{\kappa - 1}{\kappa + 1}$ with $\kappa = L/\mu$. Deterministic, geometric, boring in the good way.

A stochastic gradient over a batch B is unbiased with covariance $\mathrm{Cov}(\hat{g}) = \Sigma/|B|$, and the recurrence picks up a noise term that does **not** vanish as $w_t \to w^\star$. For a constant step the iterate converges to a stationary distribution rather than a point, with

$$\limsup_{t \to \infty}\ \mathbb{E}\|w_t - w^\star\|^2 \ = \ O\!\left(\frac{\eta\ \mathrm{tr}\,\Sigma}{\mu\,|B|}\right) \quad \Longrightarrow \quad \|w_t - w^\star\| \sim \sqrt{\eta/|B|}.$$

That is the *variance floor*: a ball around the optimum whose radius shrinks only if η shrinks. The Robbins–Monro conditions $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$ are what collapse the ball to a point, and

the schedule implemented here, $\eta_t = \eta_0/(1+\gamma t)$, satisfies both (harmonic series diverges, its squares converge).

Logistic regression. Bernoulli likelihood with $\sigma(z) = (1+e^{-z})^{-1}$ gives the average negative log-likelihood

$$\mathcal{L}(w,b) = -\tfrac{1}{n}\sum_i \Big[y_i \log\sigma(z_i) + (1-y_i)\log\big(1-\sigma(z_i)\big)\Big], \qquad z_i = w^\top x_i + b,$$

and the identity $\sigma' = \sigma(1-\sigma)$ collapses the chain rule to

$$\nabla_w \mathcal{L} = \tfrac{1}{n} X^\top(\sigma(z) - y),$$

the same shape as least squares with $\sigma(z)$ in place of the linear prediction — both are canonical-link GLMs, which is precisely why one optimizer serves both. The Hessian $\frac{1}{n}X^\top S X$ with $S = \operatorname{diag}\big(\sigma_i(1-\sigma_i)\big) \succeq 0$ makes the problem convex, and strictly so when X has full rank and no $\sigma_i \in \{0,1\}$. (On perfectly separable data no minimizer exists — $\|w\| \to \infty$ drives the loss to zero — which is what the L2 term quietly prevents.)

What we measure, and why

- **Test RMSE** (root-mean-square error on held-out data) — the typical size of a prediction error, in the units of y. It is the number a user of the model would care about. Here the noise in the data sets a floor of about 0.30: no model can beat the noise it was generated with.
- $\|w - w^\star\|$ — the distance between the learned weights and the exact least-squares solution. This is the more sensitive probe: two weight vectors can have nearly identical RMSE while one is still wandering, and the wandering is exactly what the variance-floor argument predicts. It is the measurement that tests the theory.
- **Slope of** $\log\|w-w^\star\|$ **against** $\log|B|$ — how the wandering shrinks as the batch grows. Theory says $-\frac{1}{2}$; measuring the slope rather than eyeballing curves turns that into a pass/fail check.

What I built

One optimizer, two losses. Three details worth keeping:

- **The bias is a separate parameter**, not a column of ones, for the reason derived above. The test asserts it: ridge halves $\|w\|$ while the intercept stays at 5.0.
- **The sigmoid is branch-split by sign** — $1/(1+e^{-z})$ for $z \geq 0$ and $e^z/(1+e^z)$ otherwise — so `exp` never sees a large positive argument. At ± 800 the naive form overflows; this one returns 0 and 1.
- **The closed form is solved, not inverted**: `np.linalg.solve` on the centred normal equations.

Results

Setup: test RMSE and distance to the true weights, seeds 0–2.

schedule	test RMSE (seed 0)	$\|w - w^\star\|$ median	with constant step
full batch	0.3031	0.023	0.023
mini-batch (32)	0.3109	0.051	**0.214**
SGD (1)	0.3070	0.037	**0.499**
normal equation	0.3031	0.023	—

Full-batch descent reproduces the closed-form solution to four decimals (`weight_error` 0.01794 against the normal equation's 0.01794 on seed 0 — the same number, not a coincidence: same objective, one is just slower about it).

What surprised me

The noisy schedules *do not converge at all* with a constant step size — and the theory above says exactly how badly. Measuring $\|w - w^\star\|$ against batch size at fixed η, over $|B| \in \{1, 2, \dots, 128\}$ and three seeds, the fitted slope is

$$\frac{d \log \|w - w^\star\|}{d \log |B|} = -0.54 \qquad (\text{theory: } -\tfrac{1}{2}),$$

so the floor really does scale as $|B|^{-1/2}$. I had expected mini-batch to simply converge *faster* per epoch, since it takes 15× more updates per epoch; instead the comparison is meaningless until the step schedule is part of the setup. Decay it, and all three schedules land on the closed form together.

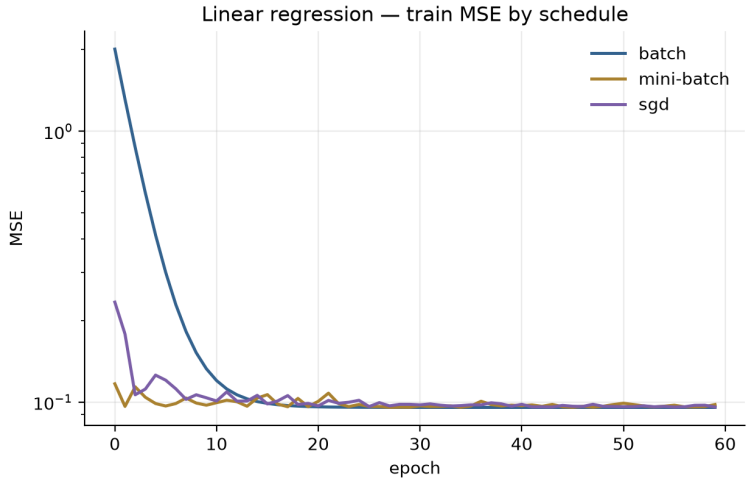


Figure 2: Linear convergence by schedule

Open threads

Momentum and adaptive steps are exactly the B5 bake-off. Logistic regression has a Newton step — IRLS, $w \leftarrow w - H^{-1}\nabla$ with the $X^\top S X$ above — that converges in a handful of iterations; worth implementing to watch second-order information pay off before it reappears as XGBoost's Newton leaf values in A4. Multiclass softmax is a small extension of the same loop.

Further reading

ESL ch. 3–4; CS229 notes 1 (supervised learning, GLMs).

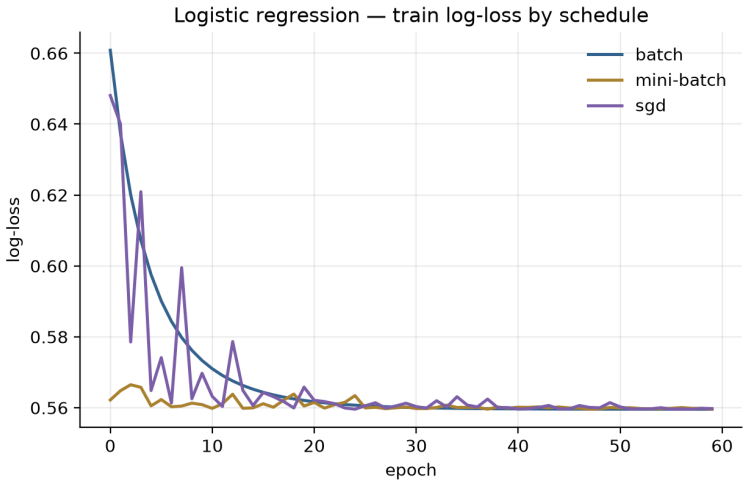


Figure 3: Logistic convergence by schedule



Figure 4: Logistic decision boundary

A2 — Decision tree and random forest

Question. Trees are the one model here with no continuous parameters at all — how much does the split *criterion* matter, and how much is really about depth? And does bagging buy back what depth costs?

The idea

A **decision tree** predicts by asking a sequence of yes/no questions about the input — *is* $x_3 \le 0.7$*? then is* $x_1 \le -1.2$*?* — until it reaches a leaf, which holds the answer (the majority class, or the mean target). Training picks, at every node, the single question that best separates the data below it, and recurses. The result carves the input space into axis-aligned rectangles. Trees are easy to read and need no feature scaling, but a deep tree keeps splitting until every leaf is nearly pure — which means it memorises the noise too.

A **random forest** fixes that by growing many deep trees, each on a different *bootstrap* resample of the data (drawn with replacement) and each allowed to consider only a random subset of features at every split, then averaging their votes. Every individual tree is still overfit, but they overfit *differently*, and averaging cancels the parts they disagree on. That is **bagging** (bootstrap aggregating).

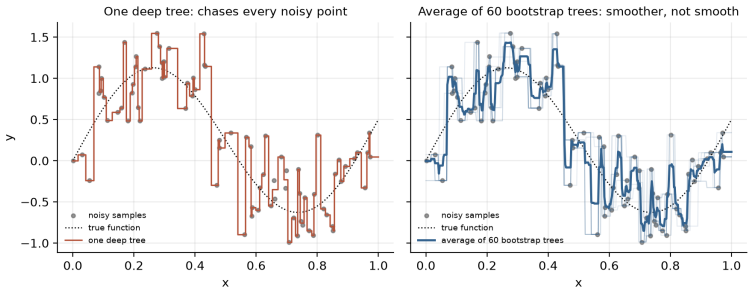


Figure 5: One deep tree against the average of many

Left: one depth-10 regression tree on 80 noisy points — a staircase through every point. Right: 60 such trees on bootstrap resamples (faint) and their average (bold), which is closer to the truth. It is still not smooth, and that is the second lesson: with a single input feature there is nothing for feature-subsampling to decorrelate, so the trees stay similar and the average hits the $\rho\sigma^2$ *floor derived below. On the 2-D spirals the forest has features to subsample and does better.*

The math

Impurity and gain. For a node with class distribution p,

$$G(p) = 1 - \sum_k p_k^2 = \sum_k p_k(1 - p_k), \qquad H(p) = -\sum_k p_k \log_2 p_k.$$

Gini is the variance of the one-hot class indicator (equivalently, the probability that two independent draws from the node disagree); entropy is expected surprisal, and its gain is the mutual information $I(Y; \mathbf{1}[x_j \le \tau])$. A split (j, τ) is scored by

$$\Delta = I(p) - \frac{n_L}{n} I(p_L) - \frac{n_R}{n} I(p_R).$$

Because $p = \frac{n_L}{n} p_L + \frac{n_R}{n} p_R$ and both G and H are strictly concave on the simplex, Jensen gives $\Delta \geq 0$ always, with equality iff $p_L = p_R$. **Every split weakly helps** — which is why the tree needs external stopping rules rather than an internal one, and why depth, not the criterion, is the real hyper-parameter.

Regression splits, and the identity the implementation runs on. With $I = \mathrm{Var}$, the leaf constant is the mean ($\arg\min_c \sum (y_i - c)^2 = \bar{y}$), and the weighted reduction telescopes:

$$n \,\mathrm{Var}(y) - n_L \,\mathrm{Var}(y_L) - n_R \,\mathrm{Var}(y_R) \;=\; \frac{\left(\sum_L y\right)^2}{n_L} + \frac{\left(\sum_R y\right)^2}{n_R} - \frac{\left(\sum y\right)^2}{n}.$$

Everything on the right is a *cumulative sum*. Sort the feature once, take `cumsum`, and every threshold is scored in one vectorized pass: $O(dn \log n)$ per node instead of the naive $O(dn^2)$. This identity returns verbatim as XGBoost's gain formula in A4.

Why bagging works. For B identically distributed trees with variance σ^2 and pairwise correlation ρ,

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Averaging only kills the second term; the floor $\rho\sigma^2$ is why `max_features` $= \sqrt{d}$ exists — decorrelating the trees attacks the term that $B \to \infty$ cannot. Bias is essentially unchanged (each deep tree is close to unbiased), so the entire gain is variance reduction.

Why out-of-bag is pessimistic, not just noisy. A bootstrap sample misses a given point with probability

$$\left(1 - \tfrac{1}{n}\right)^n \xrightarrow[n\to\infty]{} e^{-1} \approx 0.368,$$

so each OOB prediction averages $\approx 0.368B$ trees. By the variance formula above, an ensemble of $0.368B$ trees is strictly worse than one of B — the OOB score measures a forest roughly a third the size of the one you ship. The bias is downward and consistent, which is what makes it usable for model selection.

What we measure, and why

- **Test accuracy** — fraction of held-out points classified correctly. The spirals are deliberately noisy, so the ceiling is well below 1.
- **Train vs test accuracy across depth** — the *gap* between them is the overfitting made visible. A model that fits the training set better and the test set worse is fitting noise.
- **Out-of-bag (OOB) accuracy** — each tree never saw about 37% of the data, so those points are a free validation set. It is worth measuring because it is how forests are tuned in practice without a separate split, and because the math predicts it is *biased low*, which is checkable.
- **forest − tree** — the gain from bagging alone, measured over three seeds because its size turned out to be seed-dependent.

What I built

One `DecisionTree` covering both tasks — Gini and entropy for classification, variance reduction for regression — because the search is identical once a partition can be scored. That is what lets A4 embed this class as its weak learner instead of reimplementing one. Candidates are restricted to positions where the sorted feature value changes and both sides clear `min_samples_leaf`. The forest adds bootstrap resampling plus a $\sqrt{d}$ feature subset per split, and scores itself out-of-bag.

Results

Setup: spirals, 3 classes, `noise=0.7` — deliberately hard.

model	test accuracy (seed 0)	median (seeds 0–2)
tree, depth 12, Gini	0.760	0.760
tree, depth 12, entropy	0.780	0.787
tree at its best depth	0.847 (depth 5)	depth 3–8
forest, 80 trees	0.887	0.860
forest out-of-bag estimate	0.836	0.851

`forest - tree` is **+0.073 median (+0.007 to +0.107)**. Bagging reliably helps; *how much* is itself seed-dependent, which is the honest version of the claim.

What surprised me

Two things. First, Gini versus entropy is a coin flip (0.76 vs 0.78 on seed 0, and they swap on other seeds) — consistent with the concavity argument, which says both are valid concave impurities and neither has a claim to the *right* partition; depth swings accuracy by 8 points and the criterion by under 2. Second, the OOB estimate is measurably pessimistic (0.836 vs a 0.887 test score), in the direction the e^{-1} argument predicts; an 80-tree forest is OOB-scored as roughly a 29-tree one. The gap is somewhat larger than tree count alone accounts for, so the counting argument is a mechanism, not a complete model.

The depth sweep is the clean bias-variance picture: train accuracy climbs to 0.938 and stops (the tree runs out of splits), while test peaks at depth 5 and falls back, opening a 17-point gap.

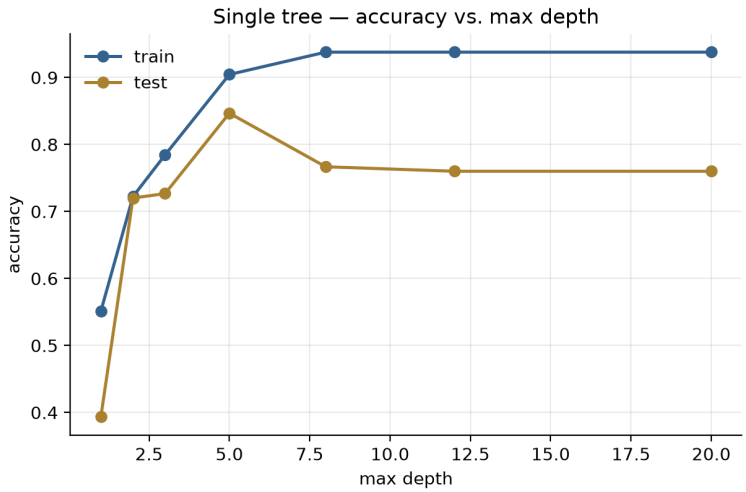


Figure 6: Depth sweep

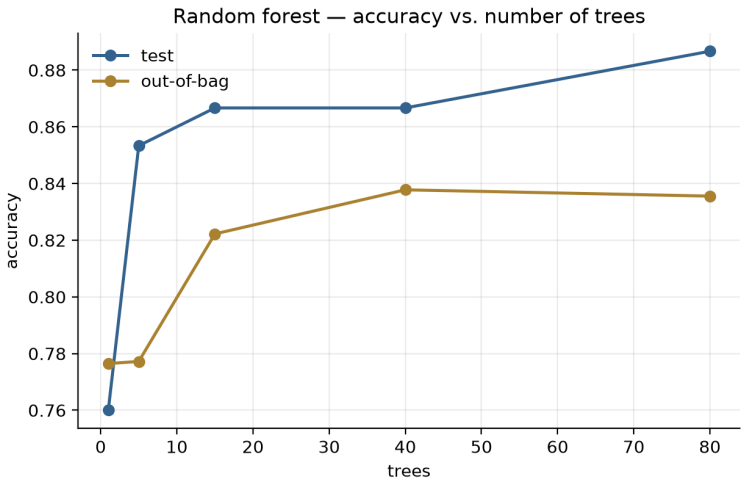


Figure 7: Forest size vs OOB

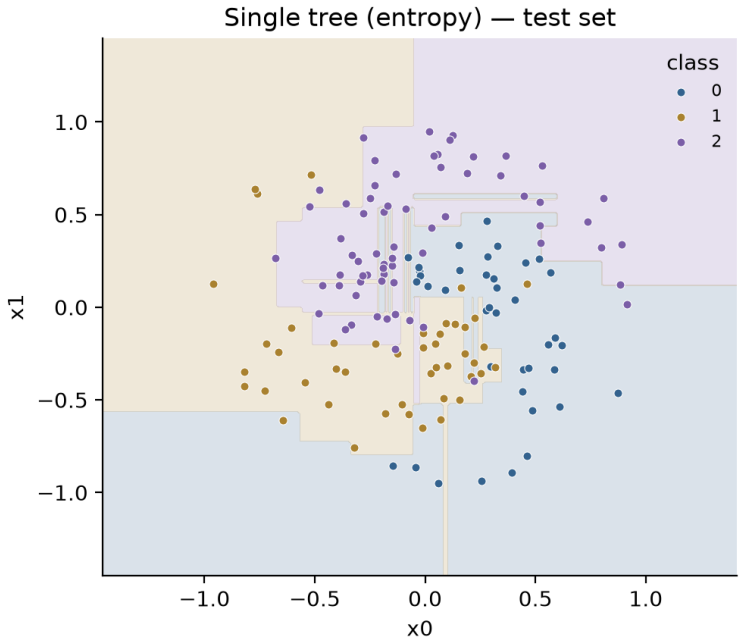


Figure 8: Single tree boundary

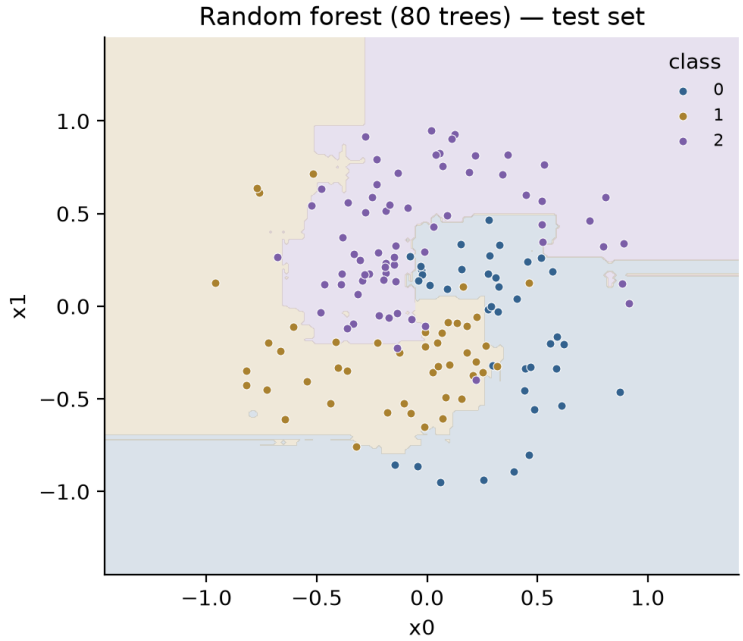


Figure 9: Forest boundary

The two boundary plots are the whole argument in one pair of images: the single tree carves the plane into confident rectangles that chase individual noisy points; the forest's averaged boundary is visibly smoother and follows the spiral arms.

Open threads

Cost-complexity pruning — minimizing $\sum_{\text{leaves}} n_m I_m + \alpha|T|$ — as the principled alternative to a depth cap. Extremely randomized trees (random thresholds rather than best ones) to test how much of the forest's gain is the ρ term rather than the fit. And the known bias of impurity-gain importances toward high-cardinality features, which A4's importance plot quietly depends on.

Further reading

Random Forests (Breiman 2001); ESL ch. 9, 15.

A3 — k-means and PCA on one dataset

Question. Both are unsupervised, both minimize squared reconstruction error, and both are usually taught apart. Run them on the same cloud: what does each actually recover?

The idea

Both algorithms here look at data **without labels** and try to summarise it.

k-means summarises a cloud of points by k representative points (centroids). It alternates two steps until nothing changes: assign every point to its nearest centroid, then move each centroid to the mean of the points assigned to it. The result is a partition of the data into k clusters.

PCA (principal component analysis) summarises the cloud by the few *directions* along which it varies most. The first principal component is the line that the points spread along the most;

the second is the best direction perpendicular to it; and so on. Keeping the top k components compresses each point to k numbers — its coordinates along those directions — with as little information lost as a linear method can manage.

They answer different questions ("which group?" versus "which directions matter?") but they are graded by the **same** yardstick: replace every point by its summary and measure how far it moved.

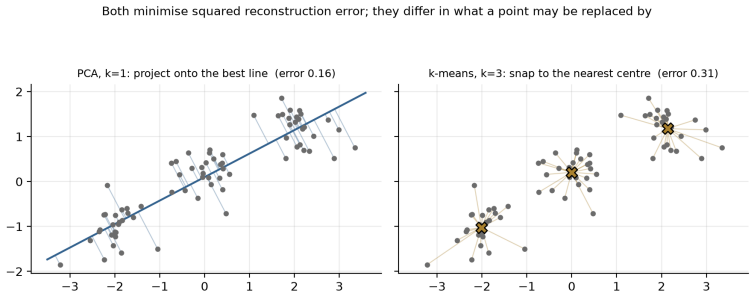


Figure 10: PCA and k-means, the same points compressed two ways

PCA may replace a point by any point on a line (left: short perpendicular drops). k-means may only replace it by one of k centres (right: every point snaps to a cross). The discrete choice is the reason k-means always reconstructs worse than PCA with $k-1$ components, which the sandwich inequality below makes exact.

The math

k-means is alternating minimization. The objective over assignments $c : \{1..n\} \to \{1..k\}$ and centres μ,

$$\phi(c, \mu) = \sum_{i=1}^{n} \|x_i - \mu_{c(i)}\|^2,$$

is minimized by Lloyd's algorithm as exact coordinate descent: with μ fixed, $c(i) = \arg\min_k \|x_i - \mu_k\|^2$; with c fixed, stationarity $\sum_{i \in C_k} (x_i - \mu_k) = 0$ gives $\mu_k = \text{mean}(C_k)$. Both steps are non-increasing and there are finitely many partitions, so the iteration terminates — at a *local* minimum, which is why restarts and seeding carry the whole burden. k-means++ draws each new centre with probability proportional to $D(x)^2$, the squared distance to the nearest chosen centre, and buys the guarantee $\mathbb{E}[\phi] \le 8(\ln k + 2)\,\phi_{\text{OPT}}$.

PCA is the Eckart–Young theorem. Minimizing reconstruction error over rank-k orthogonal projections,

$$\min_{V^\top V = I_k} \frac{1}{n} \sum_i \left\| (x_i - \bar{x}) - VV^\top (x_i - \bar{x}) \right\|^2,$$

is equivalent (by Pythagoras, since total variance is fixed) to *maximizing* projected variance, and both are solved by the top-k eigenvectors of $S = \frac{1}{n-1} X_c^\top X_c$. Taking the SVD $X_c = U\Sigma V^\top$ gives them without forming S — better conditioned, since squaring the matrix squares its condition number — with explained variance $\sigma_i^2/(n-1)$ and residual

$$\frac{1}{n}\sum_{i>k}\sigma_i^2.$$

Eckart–Young–Mirsky says no other rank-k linear compression does better in Frobenius norm.

The pairing. Both are the same constrained approximation problem $\min\|X-\hat{X}\|_F^2$ over different feasible sets: PCA over rank-k affine approximations, k-means over $\hat{X}=ZM$ with $Z\in\{0,1\}^{n\times k}$ one-hot — also rank $\le k$, but *discrete*. The k centroids span an affine subspace of dimension $\le k-1$ through the data mean, and k-means lands each point on one of k points of that subspace rather than projecting orthogonally onto the best one, which forces the sandwich

$$E_{\text{k-means}}(k)\ \ge\ E_{\text{PCA}}(k-1)\ \ge\ E_{\text{PCA}}(k).$$

Measured on this dataset, it holds at every k tested (e.g. $k=4$: $2.406\ge 1.754\ge 0.929$). Ding & He (2004) sharpen the connection: the continuous relaxation of the k-means indicator matrix is spanned by the principal components — PCA *is* relaxed k-means.

Why purity misleads and ARI does not. For a contingency table n_{ij} between true classes and clusters, with row/column sums a_i, b_j,

$$\text{ARI}=\frac{\sum_{ij}\binom{n_{ij}}{2}-\dfrac{\sum_i\binom{a_i}{2}\sum_j\binom{b_j}{2}}{\binom{n}{2}}}{\dfrac{1}{2}\left[\sum_i\binom{a_i}{2}+\sum_j\binom{b_j}{2}\right]-\dfrac{\sum_i\binom{a_i}{2}\sum_j\binom{b_j}{2}}{\binom{n}{2}}}.$$

The subtracted term is the expected agreement under a hypergeometric null, so a random partition scores 0 in expectation and splitting a correct cluster is *penalized*. Purity, $\frac{1}{n}\sum_j\max_i n_{ij}$, has no such correction and is non-decreasing under refinement — it is maximized by the singleton clustering.

What we measure, and why

- **Reconstruction error** — the mean squared distance from each point to its replacement (its centroid, or its PCA projection). It is the objective both methods minimise, so comparing it is comparing them on their own terms.
- **Explained variance** — the share of the data's total spread captured by each principal component. It is how one decides how many components to keep.
- **Adjusted Rand index (ARI)** — agreement between the clusters and the true labels (which the algorithms never saw), corrected so that random grouping scores 0 and a perfect match scores 1. The correction matters: without it the score can be gamed by splitting data into many tiny clusters.
- **Purity** — the fraction of points belonging to their cluster's majority class. It is reported *because* it is misleading: it never decreases as k grows, and seeing it reward the wrong answer is the lesson.

What I built

k-means with k-means++ seeding, Lloyd iterations to a relative tolerance, and `n_init` restarts keeping the lowest inertia; an emptied cluster is re-seeded at the worst-fit point. PCA via SVD with a sign convention so components are reproducible across runs.

Results

Setup: 4 gaussian blobs in 8-D, standardized.

k	k-means reconstruction	PCA(k) reconstruction	PCA(k-1)	ARI vs true labels
2	5.692	3.293	5.152	+0.495
3	4.000	1.754	3.293	+0.713
4	2.406	0.929	1.754	**+1.000**
6	2.115	0.269	0.505	+0.827

ARI peaks exactly at the true k on every seed, and reaches 1.000 — a perfect partition. Clustering the 2-D PCA projection instead of the raw 8-D features costs almost nothing (ARI 0.960 on seed 0, 0.991 median) for a 4× dimension reduction.

What surprised me

Purity climbs monotonically with k — 1.000 at k=4, still 1.000 at k=8 — while ARI falls away from the true k. The formula above says it must; watching it confidently reward the wrong answer is what makes it stick, and the unit test now pins the pathological case (purity 1.0, ARI 0.0 for one-point-per-cluster). The reconstruction gap also came out wider than I expected — PCA wins by roughly 2.5× at every budget — but the sandwich inequality says the direction was never in doubt; only the size was.

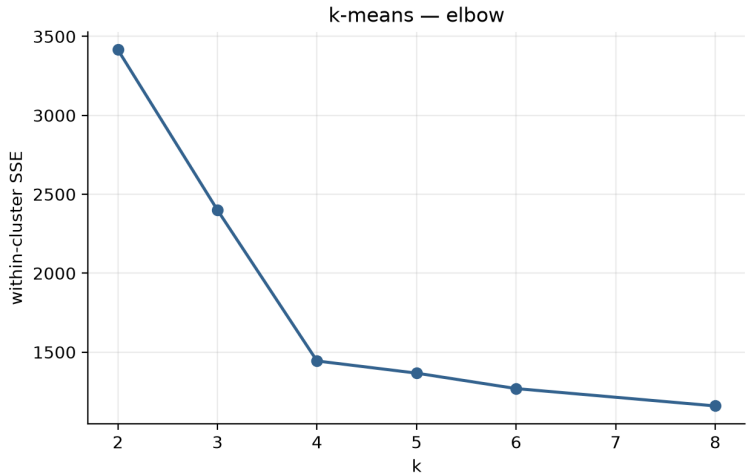


Figure 11: Elbow

Open threads

Gaussian mixtures by EM — the soft-assignment version, where a responsibility γ_{ik} replaces the hard indicator and Lloyd becomes a special case at zero covariance. Kernel PCA, which is the

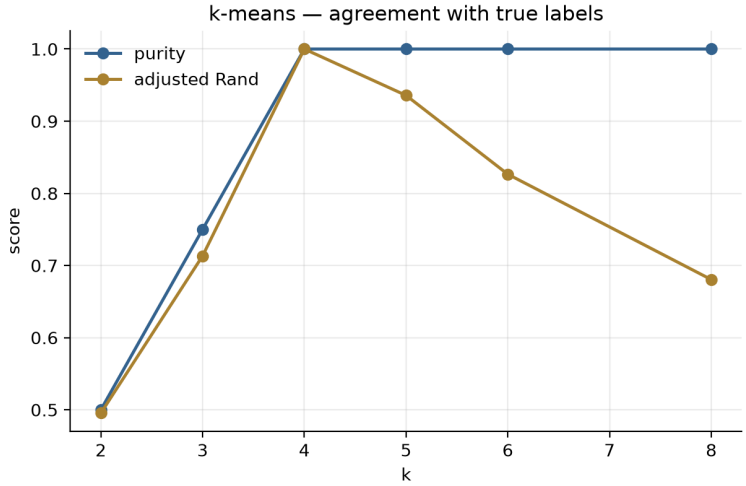


Figure 12: Agreement with true labels

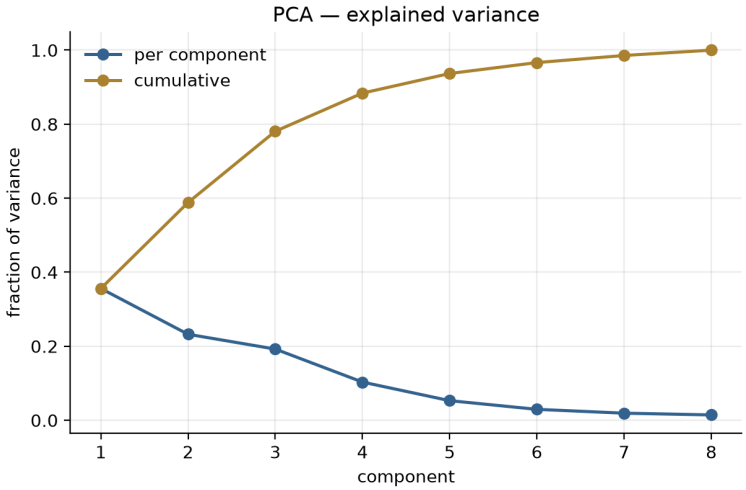


Figure 13: Explained variance

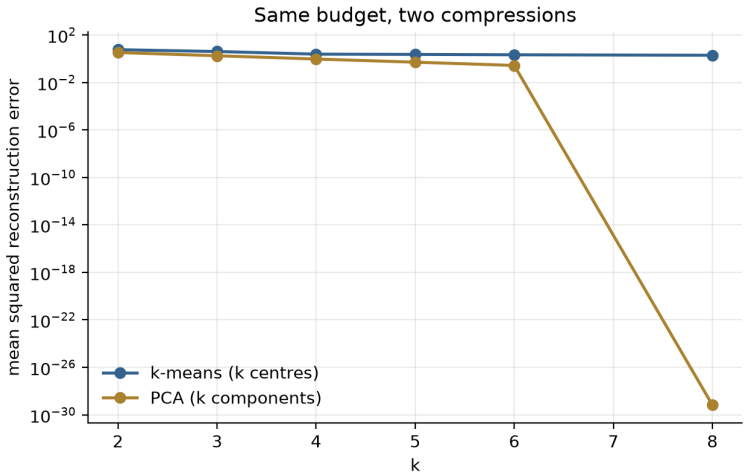


Figure 14: Reconstruction error, same budget

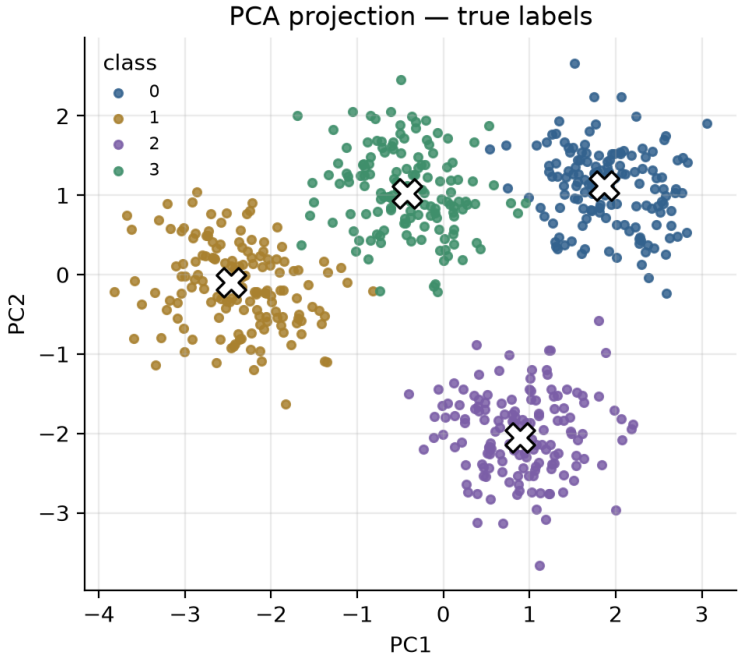


Figure 15: PCA projection, true labels

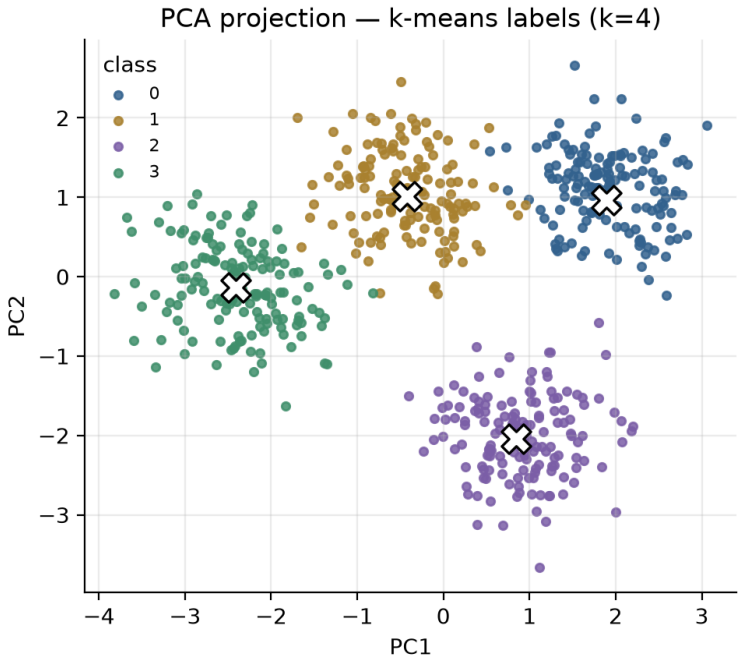


Figure 16: PCA projection, k-means labels

bridge to part D's representation learning. And choosing k honestly (silhouette, gap statistic) rather than by peeking at the labels as I do here.

Further reading

ESL ch. 14; MML book ch. 10 (PCA derived three ways).

A4 — Gradient boosting, benchmarked against XGBoost

Question. How close does a from-scratch booster get to the library everyone actually uses, given the same hyper-parameters — and where exactly does the difference come from?

The idea

Gradient boosting builds a strong model out of many weak ones, added one at a time. Start with a constant prediction (the mean of y). Look at what is still wrong — the **residuals**, $y -$ prediction. Fit a small tree (depth 3 here) to those residuals, add a *fraction* ν of it to the model, and repeat. Each new tree only has to correct the mistakes the ensemble still makes, so the model improves steadily where it is worst.

The word *gradient* is there because the residual is exactly the negative gradient of squared error with respect to the prediction. Boosting is gradient descent, but the thing being optimised is the prediction function itself, and each "step" is a tree. The fraction ν is the **learning rate** (or shrinkage): small steps generalise better but need more rounds.

XGBoost is the industrial version of the same idea. It adds a second-order (Newton) correction, regularisation on leaf values, and a great deal of engineering — histogram binning, parallel split search, C++. This rung asks which of those differences matter for accuracy.

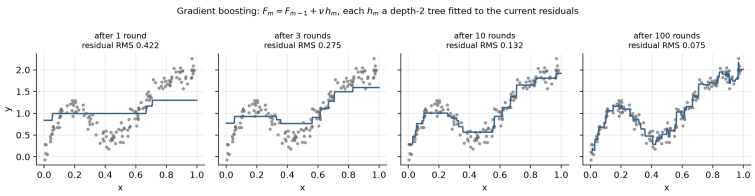


Figure 17: Boosting, round by round

One round is a single depth-2 tree: a crude staircase. By round 10 the ensemble follows the shape, and by round 100 the residual is near the noise level. Every round fits the error left by the ones before it.

The math

Boosting is gradient descent in function space. Forward stagewise additive modelling builds $F_M = F_0 + \nu \sum_{m=1}^{M} h_m$ where each step would ideally solve $h_m = \arg\min_h \sum_i L(y_i, F_{m-1}(x_i) + h(x_i))$. Gradient boosting replaces that with a first-order surrogate: compute the functional gradient at the current predictions,

$$g_i = \left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)}, \qquad h_m \approx \arg\min_h \sum_i \big(-g_i - h(x_i)\big)^2,$$

and take a shrunken step $F_m = F_{m-1} + \nu h_m$. For squared loss $L = \frac{1}{2}(y - F)^2$ the negative gradient is just the residual, $-g_i = y_i - F_{m-1}(x_i)$, and the initial constant is $F_0 = \arg\min_c \sum_i L(y_i, c) = \bar{y}$. That is the whole algorithm: fit a tree to the residuals, add a fraction of it, repeat.

The second-order version — and why our first-order one matches it here. XGBoost expands the loss to second order with $h_i = \partial^2_F L$, and adds leaf regularization:

$$\tilde{\mathcal{L}}^{(m)} = \sum_i \left[g_i f(x_i) + \tfrac{1}{2} h_i f(x_i)^2 \right] + \gamma T + \tfrac{\lambda}{2} \sum_{j=1}^{T} w_j^2,$$

which is a quadratic in each leaf weight and therefore solvable in closed form:

$$w_j^\star = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \qquad \text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma.$$

Now substitute squared loss, for which $h_i \equiv 1$, and set $\lambda = \gamma = 0$:

$$w_j^\star = \frac{\sum_{i \in I_j} r_i}{n_j} = \overline{r}_{I_j}, \qquad \text{Gain} = \frac{1}{2} \left[\frac{(\sum_L r)^2}{n_L} + \frac{(\sum_R r)^2}{n_R} - \frac{(\sum r)^2}{n} \right].$$

The leaf weight is the mean residual — exactly what a regression tree puts in its leaves — and the gain is one half of **the variance-reduction identity from A2**, so it selects the same split. Under squared loss with no regularization, the two implementations are not similar algorithms; they are *the same algorithm*, and any difference must come from defaults, arithmetic, or engineering.

That is a testable claim, so it is now a test. Matching the defaults that differ — `min_child_weight = min_samples_leaf = 5`, `base_score` $= \bar{y}$, exact splits, $\lambda = 0$ — the two implementations agree on held-out predictions to

$$\max_i |\hat{y}_i^{\text{ours}} - \hat{y}_i^{\text{xgb}}| \approx 6 \times 10^{-6}$$

on targets with standard deviation 4.7 — float32 versus float64 rounding, and nothing else.

Shrinkage. ν scales each step, so the effective complexity of F_M grows with νM: halving ν roughly doubles the rounds needed to reach the same fit, while lowering the variance of the path taken there. Since each round can only decrease training loss, test error is U-shaped in M and the location of its minimum is a function of ν — which is what the sweep below shows.

The task. Friedman #1 gives signal to five of ten features:

$$y = 10 \sin(\pi x_0 x_1) + 20 \left(x_2 - \tfrac{1}{2}\right)^2 + 10 x_3 + 5 x_4 + \varepsilon, \qquad x \sim \mathcal{U}[0,1]^{10}.$$

What we measure, and why

- **Test RMSE** — typical prediction error on held-out data, in the target's units. Friedman #1 has noise standard deviation 1, which floors it.

- **Staged test error** — RMSE after each boosting round. It shows the characteristic U-shape: error falls as rounds correct real structure, then rises once they start correcting noise. Where the minimum sits depends on the learning rate, which is the measurable content of "shrinkage".
- **Our prediction minus XGBoost's**, per test point — the direct test of the claim that the two are the same algorithm. Accuracy alone could agree by coincidence; point-by-point agreement to 10^{-6} cannot.
- **Fit time** — what the library buys, once accuracy is shown to be equal.
- **Split-gain importance** — how much total loss reduction each feature contributed. Friedman #1 puts signal in exactly five of ten features, so this is an end-to-end check with a known answer.

What I built

Squared-loss gradient boosting over A2's regression tree, with `subsample < 1` for the stochastic variant, and `staged_rmse` to replay the ensemble round by round. The benchmark goes through XGBoost's **native Booster API**, not `XGBRegressor` — the sklearn wrapper imports sklearn, and this part bans it. (That was not a guess; `XGBRegressor` raised `ImportError: sklearn needs to be installed`.)

Results

Setup: Friedman #1, 800 samples, 10 features of which 5 carry signal.

model	test RMSE (seed 0)	median (seeds 0–2)
tree, depth 3	3.457	3.327
tree, depth 10	3.138	—
random forest, 60 trees	2.333	2.333
ours, 150 rounds	**1.482**	**1.588**
ours + subsample 0.8	1.461	1.492
XGBoost, 150 rounds	**1.452**	**1.595**

What surprised me — and a claim I had to retract

On seed 0, XGBoost wins by 2% and it is tempting to write "we land within 2% of the library". Across seeds 0–2 the gap is -0.008 **median, range** -0.031 **to** $+0.030$: our implementation is ahead on two of three seeds. There is no measurable accuracy difference — and once the derivation above is in hand, there *cannot* be one, because at $\lambda = 0$ with squared loss the algorithms coincide. What the library buys is **9–15× faster fits** (0.75s vs 0.07s): histogram binning, threading and C++. My first writeup of this rung stated a seed-0 artifact as a finding; the multi-seed metrics file exists because of it, and the equivalence test exists so the claim cannot silently rot.

The learning-rate sweep is the clearest picture of shrinkage in the part: `lr=0.03` is still improving at round 150 (under-trained), `lr=0.1` converges cleanly, `lr=0.3` bottoms out around round 55 and then *degrades*, and `lr=1.0` never gets there at all — the U-shape the νM argument predicts, with its minimum sliding left as ν grows.

Pooled split-gain importance puts 74–79% of the gain on x_0–x_4 — exactly the five features Friedman #1 gives signal to — an end-to-end check that the tree, the boosting loop and the importance accounting all agree.

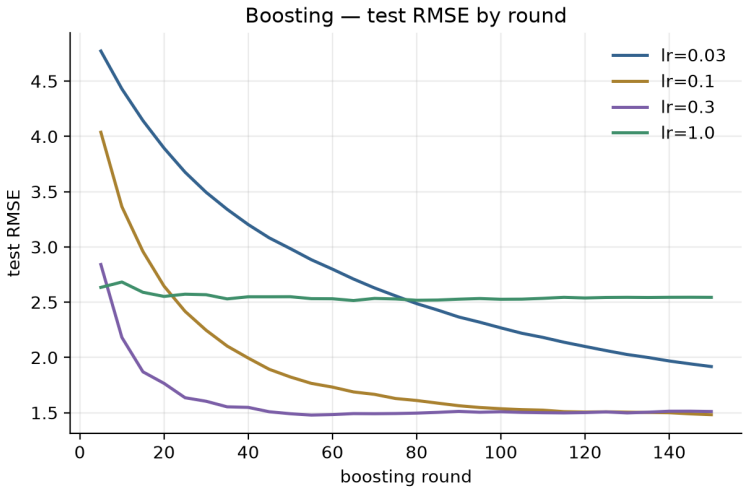


Figure 18: Staged test error by learning rate

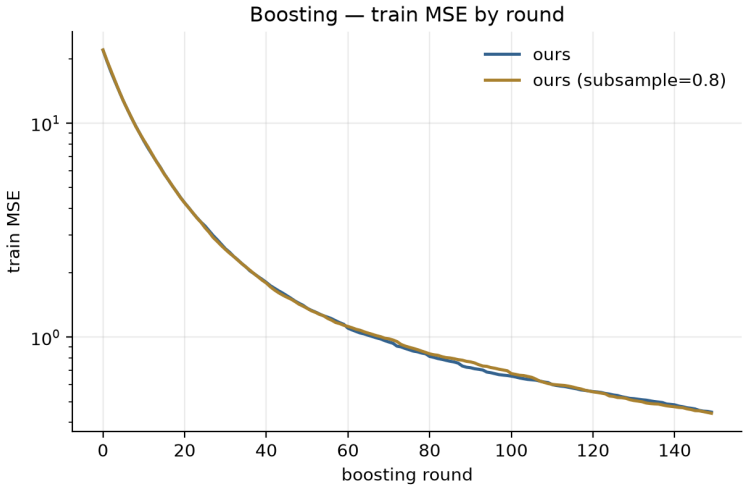


Figure 19: Train loss

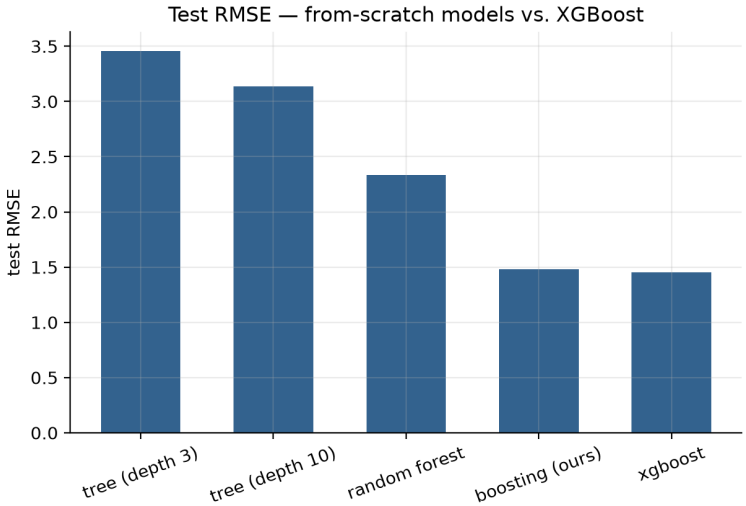


Figure 20: Model comparison

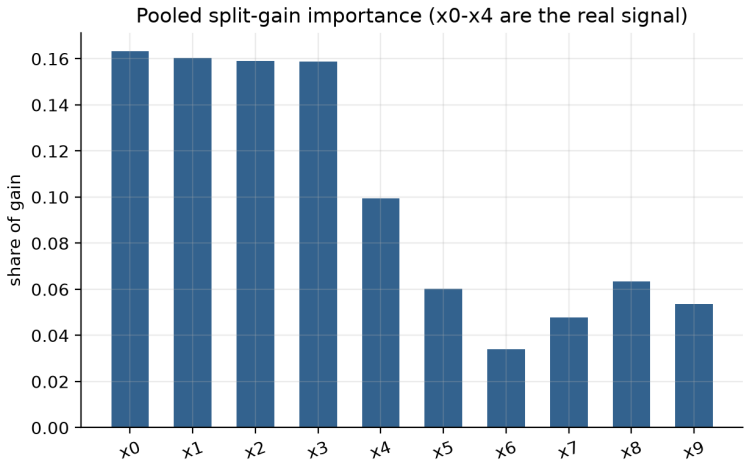


Figure 21: Feature importance

Open threads

Implement the Newton step properly — $w^\star = -\sum g / (\sum h + \lambda)$ for a loss where $h \not\equiv 1$, e.g. log-loss where $h_i = p_i(1 - p_i)$ — since that is the point at which the second-order version stops being equivalent and starts being better. Early stopping on a validation fold instead of a fixed round count. And histogram binning, to find out where the 10× actually goes.

Further reading

XGBoost (Chen & Guestrin); ESL ch. 10 (boosting as forward stagewise additive modelling).

What part A settled

1. **One objective, many search strategies.** Least squares appears four times: A1's normal equation, A2's variance-reduction splits, A3's inertia and reconstruction error, A4's residual fitting. Twice the *same identity* shows up — the cumulative-sum form of variance reduction is A2's split scan and A4's boosting gain — which is why A4 could reuse A2's tree unmodified.
2. **The math predicts the measurements, and the measurements audit the math.** The variance floor scales as $|B|^{-0.54}$ against a predicted $|B|^{-1/2}$; the k-means/PCA sandwich holds at every k; the two boosters agree to 6×10^{-6} once their defaults are aligned. Each of those was a derivation first and a number second.
3. **Single-seed numbers lie.** The XGBoost gap changed sign across three seeds. Every headline number now carries a range, regenerated by `tools/writeup_figures.py`.
4. **Variance is the whole game at this scale.** Depth, learning rate and batch size are all variance dials, and the two cures — averaging (bagging, restarts) and shrinkage (small steps, step decay, small ν) — recur in every rung with formulas that rhyme: $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ for bagging, $O(\eta \operatorname{tr}\Sigma / \mu|B|)$ for SGD.

Cost of the part: ~1800 lines including tests and the dataset generators, 30 tests, and a full run of all four rungs in about 21 seconds — small enough to rerun on every change, which is the property that made the multi-seed check cheap enough to adopt.

Figures regenerate with `uv run python tools/writeup_figures.py A`*; the numbers they are quoted against live in* `data/a-metrics.json`*. The part README is the short version of this document.*